

NavCore RX

Platform Reference Documentation

STM32G431 · ICM-20948 · LC86GLAMD · E220-900T22D

Firmware v5.5b | 100% tested | Open Source

Reference application: Bait Boat | Compatible: Hovercraft, Drone, Balance Bot, Rover

Synvertex Engineering

github.com/synvertex/navcore-rx

1. Introduction

NavCore RX is a compact, fully-integrated wireless control board for RC vehicles. It combines a 170 MHz STM32G431 microcontroller, a 9-axis IMU, GPS receiver, and a 900 MHz LoRa radio on a single tested PCB. The firmware is production-quality — derived from a working bait boat system (v5.5b) — and is structured so that users building different vehicles only need to change two functions and a configuration file.

Tested applications include bait boats, and the architecture is designed to support hovercrafts, balancing robots, drones, crop sprayers, and ground rovers with minimal firmware changes.

1.1 What This Document Covers

- Hardware overview and pin assignments
- Step-by-step wiring for each component
- How the firmware works — each subsystem explained
- Communication protocol between TX and RX boards
- Configuration reference — every tunable parameter
- Application guides — exactly what to change per vehicle type
- IMU calibration and sensor behaviour
- Troubleshooting common issues

1.2 How the System Works

The TX board (handheld controller with TFT display and joystick) sends commands over LoRa 900 MHz to the RX board. The RX board reads GPS and IMU sensors, controls motors and servos, and sends telemetry back to the TX. All communication is text-based ASCII over the LoRa link — readable and easy to extend.

INFO: The reference firmware uses the E220-900T22D radio module, but any radio module with the same 6-pin interface (RX, TX, M0, M1, AUX, GND) will work. The TX and RX boards must use the same radio module and be configured to the same channel.

2. Hardware Overview

2.1 Components on the RX Board

Component	Part Number	Interface	What It Provides
Microcontroller	STM32G431CBT6	—	170 MHz Cortex-M4, 128 KB Flash, 32 KB RAM, runs all firmware
9-Axis IMU	ICM-20948	I2C (400 kHz)	Accelerometer, gyroscope, temperature — 3 axes each
Magnetometer	AK09916 (inside ICM)	I2C via IMU	3-axis compass, tilt-compensated heading
GPS Module	LC86GLAMD	UART 115200	Position, speed, course, satellite count, HDOP
LoRa / Radio Module	E220-900T22D (reference) — any module with RX, TX, M0, M1, AUX, GND	UART	Receives commands from TX board, sends telemetry back. Reference firmware uses E220-900T22D.

2.2 System Clock

The STM32G431 runs at 170 MHz using the HSE (external crystal) through the PLL. This is the maximum rated speed for the G431 and provides precise timer periods for 50 Hz PWM generation.

System Clock Configuration	
HSE Input:	8 MHz crystal
PLL Config:	PLLM=4, PLLN=85, PLLR=2
SYSCLK:	$(8/4) \times 85 / 2 = 85 \text{ MHz} \times 2 = 170 \text{ MHz}$
APB1/APB2:	170 MHz (no divider)
TIM1/TIM2:	170 MHz clock $\rightarrow$ 50 Hz PWM via ARR=19999, PSC=169

2.3 Power Requirements

Rail	Voltage	Current (typical)	Supplied by
VCC (MCU core)	3.3 V	50 mA	LDO on board from 5V
VIN (board input)	5 V	200 mA max	BEC from ESC or separate 5V UBEC
GPS VCC	3.3 V	20 mA	Board 3.3V rail
IMU VCC	3.3 V	5 mA	Board 3.3V rail
E220 VCC	3.3 V	120 mA peak (TX)	Board 3.3V rail

Motors / ESC	7.4–11.1 V	Application dependent	Main battery (direct)
--------------	------------	-----------------------	-----------------------

NOTE: Always power the RX board from a dedicated BEC (Battery Eliminator Circuit) or a separate 5V regulated supply. Do not power it directly from the main battery without regulation.

3. Pin Assignments and Wiring

3.1 Complete GPIO Map

STM32 Pin	Function	Direction	Peripheral	Notes
PA0	Servo 1 PWM output	Output	TIM2 CH1	50 Hz, 500–2500 μ s
PA1	Front light MOSFET	Output	GPIO	High = light ON
PA2	GPS UART TX	Output	USART2 TX	To GPS RX pin
PA3	GPS UART RX	Input	USART2 RX	From GPS TX pin
PA8	ESC Left PWM	Output	TIM1 CH1	50 Hz, 1000–2000 μ s
PA9	ESC Right PWM	Output	TIM1 CH2	50 Hz, 1000–2000 μ s
PA10	Servo 2 PWM output	Output	TIM1 CH3	50 Hz, 500–2500 μ s
PB1	Battery voltage ADC	Analog In	ADC1	Via 33k/3.3k divider
PB5	Rear light MOSFET	Output	GPIO	High = light ON
PB6	I2C1 SCL	Output	I2C1	To IMU SCL
PB7	I2C1 SDA	Bidirectional	I2C1	To IMU SDA
PB10	E220 UART TX	Output	USART3 TX	To E220 RX pin
PB11	E220 UART RX	Input	USART3 RX	From E220 TX pin

NOTE: Pin assignments for USART2 (GPS) and USART3 (Radio) may differ depending on your board revision. Always verify against your CubeMX .ioc file before wiring.

3.2 Wiring the ESC / Motors

Both ESCs receive standard RC PWM signals (50 Hz, 1000–2000 μ s pulse width). Each ESC has three wires: signal, 5V, and ground.

ESC Wire	Connect to	Signal Details
Left ESC — Signal	PA8 (TIM1 CH1)	50 Hz PWM. Idle=1000 μ s, Full throttle=2000 μ s. Starts moving at ~1200 μ s.
Right ESC — Signal	PA9 (TIM1 CH2)	50 Hz PWM. Idle=1000 μ s, Full throttle=2000 μ s. Starts moving at ~1370 μ s.
ESC 5V (red)	Not connected or BEC output	ESC can power the RX board via BEC — connect only one ESC's BEC
ESC Ground	Board GND	All grounds must share a common reference

INFO: The left and right ESC start pulses (MOTOR_LEFT_START and MOTOR_RIGHT_START in config.h) are different for each ESC because motor characteristics vary. Set these values to the minimum pulse that makes the motor spin without stalling.

ESC Calibration Procedure

1. Power on the RX board with no motor command (serial terminal open)
2. Observe console: '[ESC] Armed at 1000' confirms idle signal is active
3. With joystick at zero, motor should not spin
4. Increase MOTOR_LEFT_START in config.h until the left motor just starts to spin reliably
5. Repeat for MOTOR_RIGHT_START on the right motor
6. Typical values: 1100–1400 μ s depending on ESC brand

3.3 Wiring the Servos

The board supports two servo outputs. Servo 1 (PA0/TIM2) is used for the bait drop mechanism in the reference firmware. Servo 2 (PA10/TIM1 CH3) is available for a second function.

Servo Wire	Connect to	PWM Range
Servo 1 — Signal	PA0 (TIM2 CH1)	500 μ s = 0°, 1500 μ s = 90°, 2500 μ s = 180°
Servo 2 — Signal	PA10 (TIM1 CH3)	Same range. Moves in sync with Servo 1 in reference firmware.
Servo VCC (red)	5V (from BEC)	Most servos require 5V, not 3.3V
Servo GND (brown)	Board GND	Shared ground with board

The 4-step bait drop sequence in the reference firmware:

Bait Drop Servo Sequence	
Step 0:	500 μ s → 0° (fully closed / reset position)
Step 1:	1500 μ s → 90° (partial open)
Step 2:	2500 μ s → 180° (fully open — bait drops)
Step 3:	1500 μ s → 90° (return to centre)
Each press of the bait button advances one step.	
To change positions: edit servo_positions[] in Servo_Step()	

3.4 Wiring the GPS (LC86GLAMD)

The LC86GLAMD is a compact GPS module that communicates via UART at 115200 baud. It uses 3.3V logic, compatible directly with the STM32G431.

LC86GLAMD Pin	Connect to Board	Notes
---------------	------------------	-------

VCC	3.3V rail	Module operates on 3.3V. Do not connect to 5V.
GND	GND	Common ground
TX	PA3 (USART2 RX)	GPS sends NMEA sentences to MCU
RX	PA2 (USART2 TX)	MCU sends configuration commands to GPS
1PPS	Not connected	1-pulse-per-second timing output — optional, unused in firmware

The firmware configures the GPS at boot using PAIR commands:

GPS Configuration Commands (sent automatically at boot)

```
$PAIR066 — Enable GGA and RMC sentences only (reduces UART traffic)
$PAIR050 — Set update rate to 1 Hz (1000 ms interval)
$PAIR386 — Disable SBAS (simplifies parsing)
$PAIR410 — Configure antenna mode
```

INFO: The GPS takes up to 60 seconds to acquire a fix outdoors from cold start. The TX screen will show 'GPS searching...' until the LC86G reports fix quality ≥ 1 . Place the GPS antenna with a clear view of the sky.

3.5 Wiring the IMU (ICM-20948)

The ICM-20948 uses I2C at 400 kHz. Its I2C address is either 0x68 or 0x69 depending on the AD0 pin state. The firmware automatically detects both addresses at boot.

ICM-20948 Pin	Connect to Board	Notes
VDD	3.3V	Supply voltage
GND	GND	Ground
SCL	PB6 (I2C1 SCL)	400 kHz I2C clock
SDA	PB7 (I2C1 SDA)	I2C data — bidirectional, needs pull-up
AD0	GND for 0x68 / 3.3V for 0x69	Sets I2C address. Firmware auto-detects both.
INT	Not connected	Data-ready interrupt — not used in firmware

NOTE: Add 4.7 k Ω pull-up resistors from SDA and SCL to 3.3V if not already present on your breakout board. Most ICM-20948 breakout boards include them. Check your board's schematic.

The AK09916 magnetometer is built into the ICM-20948 package and is accessed directly over I2C at address 0x0C. The firmware initialises it automatically when the ICM is detected.

3.6 Wiring the Radio Module

The reference firmware uses the E220-900T22D LoRa module, but any radio module with the same 6-pin interface will work — including other E220 variants, E32, E22, or any module that exposes RX, TX, M0, M1, AUX, and GND. The MCU communicates via UART and

controls the module using M0/M1 for mode selection and AUX to detect when the module is busy.

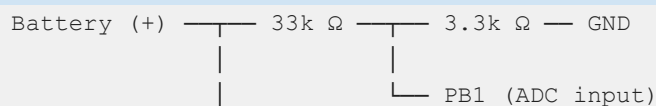
Radio Module Pin	Connect to Board	Notes
VCC	3.3V	Most LoRa modules in this family run on 3.3V. Check your module datasheet.
GND	GND	Common ground — must share with MCU and all other modules
TX	PB11 (USART3 RX)	Radio sends data to MCU — connect Radio TX to MCU RX
RX	PB10 (USART3 TX)	MCU sends data to radio — connect MCU TX to Radio RX
M0	GPIO Output (see e220.h)	Mode select pin — firmware drives this to switch between config and data modes
M1	GPIO Output (see e220.h)	Mode select pin — firmware drives this alongside M0
AUX	GPIO Input (see e220.h)	Busy indicator — HIGH means module is ready. Firmware waits for AUX HIGH before sending.

INFO: Only these 6 connections are needed: RX, TX, M0, M1, AUX, GND. The E220-900T22D is the tested reference module. If you use a different module, set it to the same UART baud rate and transparent transmission mode. Update E220_Configure() in e220.c if the AT command syntax differs.

3.7 Battery Voltage Monitoring

The battery voltage is measured on PB1 through a resistor divider that scales the battery voltage to the 0–3.3V ADC range.

Battery Voltage Divider



Divider ratio: $3.3k / (33k + 3.3k) = 1/11.09$
 ADC scale: $BAT_ADC_TO_MV = 8.912 \text{ mV per ADC count}$
 Max input: $4095 \times 8.912 = 36.5 \text{ V (suitable for 3S LiPo at 12.6V)}$
 Measured at: ADC1 channel, 16-sample average

The firmware maps ADC readings to percentage using a 3S LiPo discharge curve:

Battery Voltage	Percentage Reported
12.6 V and above	100%
11.4 V	75%
11.1 V	50%
10.5 V	25%

9.9 V	10%
9.0 V and below	0%

3.8 LED / Light Outputs

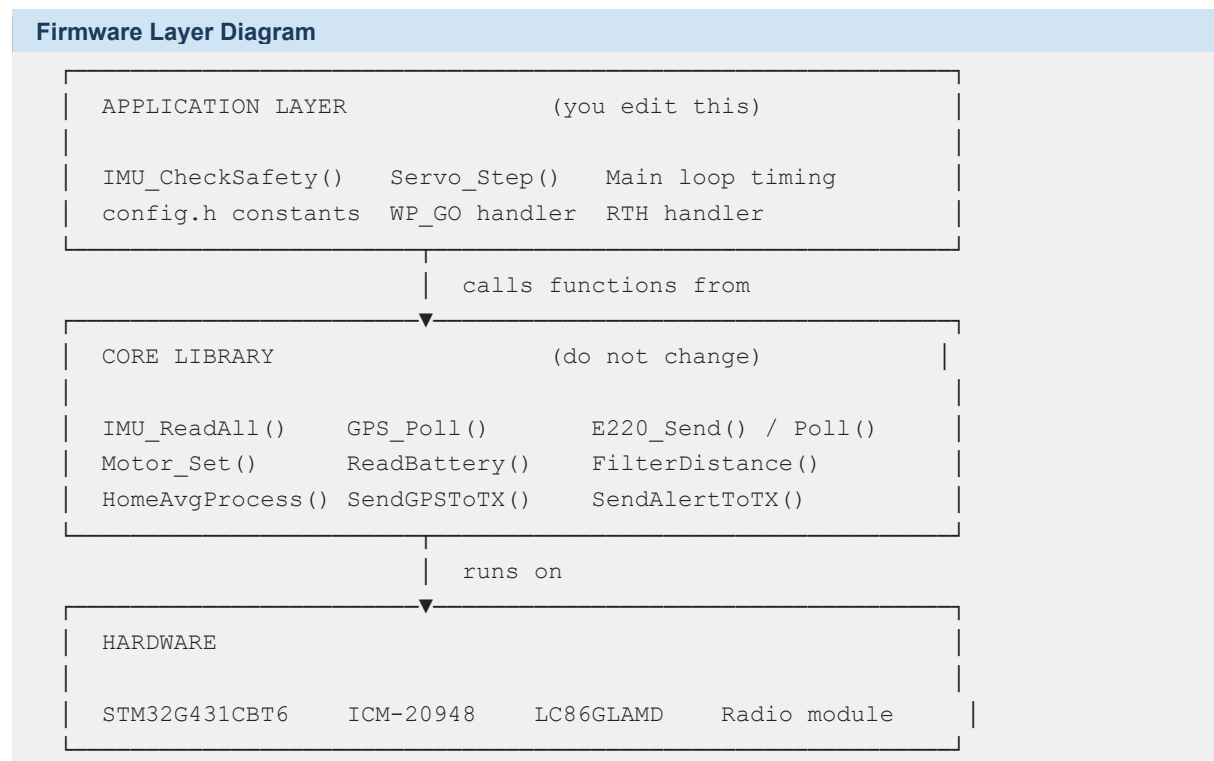
Two MOSFET-driven outputs control front and rear lights. The GPIO pins output 3.3V logic; an N-channel MOSFET switches the higher-voltage LED load.

Output	Board Pin	Control
Front light	PA1	GPIO HIGH = lights ON. Commands: LIGHT_ON / LIGHT_OFF from TX.
Rear light	PB5	Switches with PA1 — both front and rear controlled together.

4. Firmware Architecture

4.1 Two-Layer Structure

The firmware is organised into two layers. The core library handles all hardware communication — drivers for GPS, IMU, radio, and motor PWM. The application layer uses these drivers to implement the vehicle-specific control logic. Users building a different vehicle only edit the application layer.



4.2 Main Loop Task Schedule

The main loop runs every 1–2 ms (HAL_Delay(1) at the end). Tasks fire based on elapsed time using HAL_GetTick():

Task	Interval	What It Does
Radio receive	Every loop tick	Checks for incoming LoRa message from TX
IMU read	100 ms	Reads ICM-20948, computes pitch/roll/heading, runs safety check
GPS parsing	As received	Interrupt-driven ring buffer — parsed every loop tick
GPS send to TX	2000 ms (home set) / 3000 ms (pre-home)	Sends G: telemetry packet over LoRa
Status send	3000 ms (no fix)	Sends S: status packet when GPS has no fix

Battery read	30 000 ms	16-sample ADC average, updates bat_percent
Radio health	60 000 ms	Resets GPS UART if no new bytes received
Motor timeout	15 000 ms	Stops motors if no TX contact for 15 seconds
Debug print	15 000 ms	Sends status summary to SWO/ITM debug output

4.3 Core Library Function Reference

These functions are provided and should not be modified. Call them from your application layer.

Function	Returns	Description
IMU_ReadAll()	bool	Reads all 14 bytes from ICM-20948 in one burst. Returns false if I2C fails.
IMU_Process()	void	Converts raw data to pitch, roll, heading (degrees). Updates imu_pitch, imu_roll, imu_heading.
IMU_CheckSafety()	void	Classifies boat_status from sensor data. This is the function you replace per application.
MAG_Read()	bool	Reads AK09916 magnetometer. Returns false if not ready or overflow.
GPS_Poll(buf, len)	bool	Returns true + fills buf with one complete NMEA sentence from ring buffer.
GPS_ParseGGA()	void	Parses GGA sentence — updates lat, lon, fix, sats, hdop, altitude.
GPS_ParseRMC()	void	Parses RMC sentence — updates speed (km/h) and course (degrees).
E220_Poll(buf, len)	bool	Returns true if a complete LoRa message was received into buf.
E220_Send(str)	void	Sends a null-terminated ASCII string over LoRa to the TX board.
Motor_Set(l, r)	void	Sets left and right motor speed (0–100%). Handles ESC pulse mapping.
Motor_Stop()	void	Immediately sends idle pulse (1000 µs) to both ESCs.
Servo_Step()	void	Advances servo to next step in servo_positions[] array.
ReadBattery()	void	Reads PB1 ADC, updates bat_percent and bat_mv.
FilterDistance(raw)	void	Applies Kalman + EMA filter to raw GPS distance. Updates filtered_distance_m.
SendGPSToTX()	void	Builds and sends the G: telemetry packet to TX.
SendAlertToTX()	void	Sends A: alert packet with IMU status, pitch, roll, heading, temperature.

5. Communication Protocol (TX ↔ RX)

All messages are plain ASCII text terminated with a newline (\n). The radio module handles framing — each E220_Send() call delivers one atomic message. There is no binary encoding. This protocol works with any radio module configured in transparent UART mode.

5.1 TX → RX Commands

Command	Format Example	Description
Motor	M:65,72	Set left motor to 65%, right to 72%. Range 0–100 each.
Stop	STOP	Immediately stop motors and reset servo to position 0.
Servo step	SERVO_STEP:2	Move servo to step 2 (0–3). Idempotent — duplicate sends ignored.
Light on	LIGHT_ON	Activate front (PA1) and rear (PB5) lights.
Light off	LIGHT_OFF	Deactivate both lights.
Set home	SET_HOME	Start 10-sample GPS averaging to set home position.
RTH on	RTH_ON	Activate return-to-home mode (stub — navigation not yet implemented).
RTH off	RTH_OFF	Deactivate return-to-home mode.
Waypoint go	WP_GO:33,552834,N,73,123456,E	Navigate to given coordinates (stub — coordinates received, autopilot not implemented).
Reset status	RESET_CAPSIZE	Clear BOAT_CAPSIZED status back to BOAT_OK.

5.2 RX → TX Messages

Message	Format Example	When Sent
GPS packet	G:33,552834,N,73,123456,E,12,2,120,55,82,45,48,1800,15	Every 2s when home set. Every 3s pre-home.
Status	S:82,9,2	Every 3s when GPS has no fix. Format: S:bat%,sats,fix
Alert	A:1,1800,120,30,230	When boat_status changes. Format: A:status,heading,pitch,roll,temp×10
Home set	H:33,552834,N,73,123456,E	After home averaging completes. Followed by HOME_SET.
Home confirm	HOME_SET	Sent after H: packet to confirm home is stored.
Averaging	AVERAGING / AVG:5/10	Progress updates during 10-sample home GPS averaging.

No fix	NO_FIX	Sent if SET_HOME received but GPS has no fix yet.
OK	OK	Acknowledges LIGHT_ON/OFF, RTH_ON/OFF, STOP commands.
Bait confirm	BAIT:2	Confirms servo moved to step 2. Clears TX retry.
Stop confirm	STP	Confirms motors stopped when M:0,0 received.

5.3 GPS Packet Format (G:)

The G: packet carries all navigation telemetry in one comma-separated string:

G: Packet Field Definitions

G:lat_int,lat_frac,lat_dir,lon_int,lon_frac,lon_dir,sats,fix,hdop×100,spd×10,bat%,dist_m,raw_di

Example: G:33,552834,N,73,123456,E,12,2,120,55,82,45,48,1800,15

Field breakdown:

```

lat_int=33      integer degrees of latitude
lat_frac=552834 fractional part (×1000000)
lat_dir=N       N or S
lon_int=73      integer degrees of longitude
lon_frac=123456 fractional part (×1000000)
lon_dir=E       E or W
sats=12         satellites in use
fix=2           0=no fix, 1=GPS, 2=DGPS
hdop×100=120    HDOP × 100 (so 1.20 HDOP)
spd×10=55       speed × 10 km/h (so 5.5 km/h)
bat%=82         battery percentage
dist_m=45       filtered distance from home in metres
raw_dist=48     unfiltered GPS distance in metres
heading×10=1800 heading × 10 degrees (so 180.0°)
accuracy×10=15  GPS accuracy estimate × 10 metres

```

6. Configuration Reference

All tunable parameters are defined as `#define` constants. Change these in `config.h` (or at the top of `main.c`) before compiling.

6.1 Motor Parameters

Parameter	Default	Unit	Description
ESC_MIN_PULSE	1000	µs	Idle pulse — ESC armed but not spinning
ESC_MAX_PULSE	2000	µs	Full throttle pulse
MOTOR_LEFT_START	1200	µs	Minimum pulse that spins the left motor
MOTOR_RIGHT_START	1370	µs	Minimum pulse that spins the right motor
MOTOR_MIN_PERCENT	3	%	Below this percentage → idle pulse (stops motor)
MOTOR_TIMEOUT_MS	15000	ms	Stop motors if no TX contact for this long

6.2 Servo Parameters

Parameter	Default	Unit	Description
SERVO_0	500	µs	0° position
SERVO_90	1500	µs	90° position
SERVO_180	2500	µs	180° position
SERVO_STEPS	4	—	Number of bait-drop steps in the cycle

6.3 GPS and Distance Filter

Parameter	Default	Unit	Description
GPS_SEND_INTERVAL	2000	ms	How often GPS packet is sent when home is set
GPS_PRE_HOME_INT	3000	ms	GPS send interval before home is set
DEAD_ZONE_BASE_M	3.0	m	Minimum GPS noise zone (distance reads zero inside)
DEAD_ZONE_HDOP_K	1.5	—	Dead zone grows by HDOP × this factor
DEAD_ZONE_MAX_M	12.0	m	Maximum dead zone size regardless of HDOP
HYSTERESIS_M	2.0	m	Prevents flickering when near dead zone edge
KALMAN_Q_STILL	0.01	—	Kalman process noise when stationary
KALMAN_Q_MOVING	0.5	—	Kalman process noise when moving slowly
KALMAN_Q_FAST	2.0	—	Kalman process noise at high speed
HOME_AVG_SAMPLES	10	—	Number of GPS samples averaged for home position

6.4 IMU Safety Thresholds

Parameter	Default	Unit	Description
TILT_WARN_DEG	20.0	°	Pitch or roll above this → BOAT_TILT_WARN
TILT_CRITICAL_DEG	45.0	°	Above this → BOAT_TILT_CRIT
TILT_CAPSIZE_DEG	60.0	°	Above this → BOAT_CAPSIZED
ROUGH_WATER_DPS	50.0	dps	Total gyro magnitude above this → BOAT_ROUGH_WATER
STUCK_TIMEOUT_MS	3000	ms	Motors on with no IMU movement for this long → BOAT_STUCK
STUCK_GYRO_MIN	2.0	dps	Gyro total below this counts as 'not moving'
IMU_READ_INTERVAL	100	ms	How often IMU is sampled

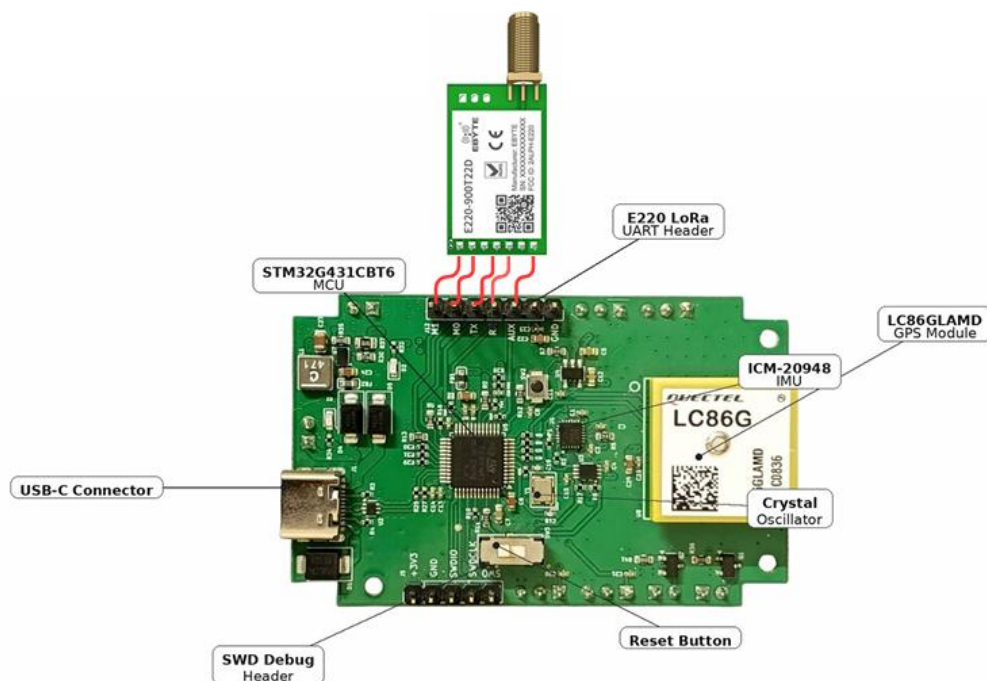
7. Application Guides

Each guide describes which functions to keep unchanged, which to modify, and what new logic to add. The reference bait boat firmware is in `/examples/bait_boat/` and is the best starting point for any application.

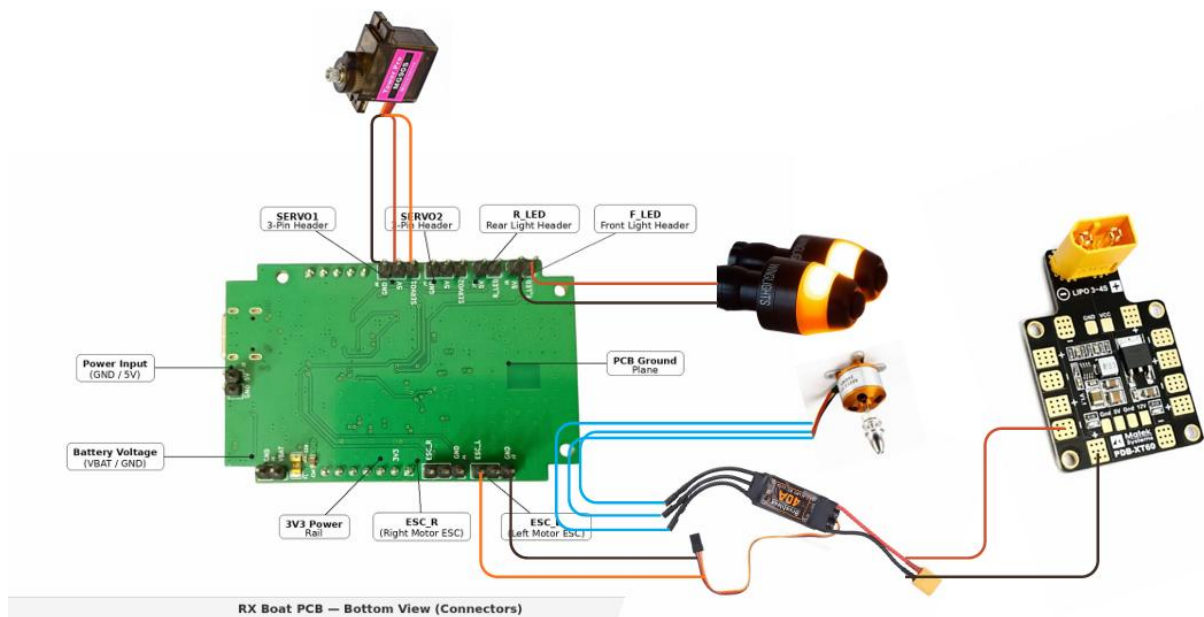
7.1 Bait Boat (Reference Application — Fully Tested)

This is the production application the board was designed and tested against. Use it as-is or as the starting point for all other applications.

Firmware element	Status	Details
Motor_Set() / Motor_Stop()	Keep	Dual ESC differential drive. Left on TIM1 CH1, right on TIM1 CH2.
Servo_Step()	Keep	4-step bait drop mechanism. Two servos move in sync.
IMU_CheckSafety()	Keep	Tilt, capsize, rough water, stuck detection all relevant for a boat.
GPS + distance filter	Keep	Kalman+EMA filter, dead zone, home averaging — all relevant.
TILT_WARN_DEG = 20°	May need tuning	Choppy water can exceed 20° on normal waves. Test and increase if needed.
Magnetometer heading	Calibrate	See Section 8.3 — motors cause magnetic interference. One-time field job.



RX Boat PCB — Top View



7.2 Hovercraft

A hovercraft uses one motor for lift (maintains the air cushion) and one for thrust. A servo drives the rudder for steering. The overall system maps directly to the existing outputs.

Firmware element	Action	What to do
Motor_Set(lift%, thrust%)	Keep	Left output → lift fan ESC. Right output → thrust fan ESC.
Servo_Step()	Change	Replace 4-step bait array with rudder sweep. Example: {1000,1250,1500,1750,2000} μ s
IMU_CheckSafety()	Change	Increase tilt thresholds — hovercrafts naturally tilt more than boats on turns.
GPS + home	Keep	Useful for range awareness and optional waypoint navigation.
GPS_SEND_INTERVAL	Optional	Can reduce to 1000 ms for faster display updates on TX.

Key change — replace the servo positions array in Servo_Step():

Hovercraft Rudder Servo Example

```
// For rudder steering (replaces bait drop sequence)
static const uint32_t servo_positions[] = {
    1500,    // Centre (straight)
    1250,    // Left 30°
    1500,    // Centre
    1750,    // Right 30°
};

// Or for proportional control, read steering from radio:
// servo_pulse = 1500 + (steering_pct * 500 / 100);
```

INFO: For hovercraft, the lift fan should auto-throttle to maintain cushion height. Consider adding a simple auto-lift mode: set lift motor to a fixed percentage (e.g. 70%) and only vary thrust from the joystick.

7.3 Self-Balancing Robot

A balancing robot uses the pitch angle from the IMU to continuously correct motor speed via a PID controller. GPS and home position are not needed.

Firmware element	Action	What to do
IMU_ReadAll() + imu_pitch	Keep — critical	Pitch angle is the core input to the PID. Read every 10 ms for balance.
Motor_Set(l, r)	Change	Needs to support reverse — modify to allow negative PWM or add H-bridge logic.
GPS, home, distance	Remove	Not needed for a ground robot. Remove GPS polling and SendGPSToTX().
IMU_CHECK_INTERVAL	Change	Reduce from 100 ms to 10 ms for fast balance loop response.
IMU_CheckSafety()	Replace	Replace with PID controller using imu_pitch. Add Kp, Ki, Kd constants to config.h.
GPS_SEND_INTERVAL	Replace	Send IMU telemetry (pitch, motor speeds) to TX instead of GPS data.

PID controller to add inside the main loop (replace IMU_CheckSafety):

Balance Robot PID Controller

```
// In config.h — tune these on your hardware
#define PID_KP    25.0f
#define PID_KI    0.5f
#define PID_KD    8.0f
#define BALANCE_SETPoint  0.0f    // target pitch angle

// In main loop (every 10 ms)
float error      = BALANCE_SETPoint - imu_pitch;
pid_integral    += error * 0.01f;    // 10 ms sample time
float derivative = (error - prev_error) / 0.01f;
float output     = PID_KP*error + PID_KI*pid_integral + PID_KD*derivative;
prev_error      = error;

// Steer with joystick on top of balance output
int steer = radio_get_ch(2);
Motor_Set((int)(output + steer), (int)(output - steer));
```

7.4 Drone (Attitude Control)

A drone requires fast attitude control using all three IMU axes. The firmware provides the raw IMU data and GPS; the user adds PID loops for roll, pitch, yaw, and optionally altitude.

Firmware element	Action	What to do
IMU_ReadAll(), accel_g[], gyro_dps[]	Keep — critical	All 6 DOF data at 100 ms. Reduce IMU_READ_INTERVAL to 2–5 ms for flight.
Motor_Set()	Expand	Need 4 ESC outputs. Add TIM1 CH4 or a second timer for the fourth motor.
IMU_Process()	Extend	Add Madgwick or complementary filter for fused roll/pitch/yaw angles.
IMU_CheckSafety()	Replace	Replace with 3-axis PID controllers (roll, pitch, yaw).
GPS	Keep — optional	Use for altitude hold and GPS position hold. Optional for basic flight.
Radio protocol	Change	TX must send 4 channels: throttle, roll, pitch, yaw. Update ParseMotorCmd() on RX.

NOTE: Drone firmware requires IMU updates every 2–5 ms, not 100 ms. Reduce IMU_READ_INTERVAL to 5 and ensure no blocking calls exist in the main loop.

7.5 Crop Sprayer / Ground Rover

A ground rover or crop sprayer uses GPS for coverage tracking and a servo or relay to control the spray pump. IMU tilt detection is less critical on flat agricultural ground.

Firmware element	Action	What to do
Motor_Set()	Keep	Standard differential drive. Works as-is.
Servo_Step()	Change	Drive a pump relay: step 0 = pump off (1000 μ s), step 1 = pump on (2000 μ s).
GPS + distance	Keep	GPS position tracking useful for coverage mapping.
IMU_CheckSafety()	Simplify	On flat ground, only check for extreme tilt (vehicle overturn). Remove rough water check.
WP_GO handler	Implement	Use CalcDistanceD() and gps_course to steer toward waypoint automatically.
GPS_SEND_INTERVAL	Reduce	Set to 1000 ms for more responsive position display on TX.

8. IMU and Sensor Guide

8.1 How the IMU Works

The ICM-20948 contains three sensors: a 3-axis accelerometer, a 3-axis gyroscope, and a 3-axis magnetometer (AK09916). All three are read together every 100 ms.

Sensor	Output	Used for
Accelerometer	accel_g[3] (units: g)	Pitch and roll angle calculation from gravity vector. Accurate when stationary.
Gyroscope	gyro_dps[3] (units: °/s)	Angular rate. Used for rough water detection and stuck detection.
Magnetometer	mag_ut[3] (units: μT)	Compass heading. Requires tilt compensation using accel pitch and roll.
Temperature	temp_c (units: °C)	Board temperature inside the IMU package. Sent in alert packets.

8.2 How Pitch, Roll, and Heading are Computed

Pitch / Roll / Heading Calculation (from IMU_Process())

```
// Pitch – forward/back tilt (degrees)
imu_pitch = atan2(accel_x, sqrt(accel_y2 + accel_z2)) × 57.2958

// Roll – side-to-side tilt (degrees)
imu_roll = atan2(accel_y, sqrt(accel_x2 + accel_z2)) × 57.2958

// Tilt-compensated heading (degrees, 0 = North)
pitchR = imu_pitch × 0.01745 // convert to radians
rollR = imu_roll × 0.01745
mx_compensated = mag_x × cos(pitchR) + mag_z × sin(pitchR)
my_compensated = mag_x × sin(rollR) × sin(pitchR) + mag_y × cos(rollR)
                - mag_z × sin(rollR) × cos(pitchR)
imu_heading = atan2(my_compensated, mx_compensated) × 57.2958
if (imu_heading < 0) imu_heading += 360
```

8.3 IMU Safety Classification (IMU_CheckSafety)

The current safety classification is designed for waterborne vehicles. The boat_status enum drives the icon displayed on the TX screen and triggers alert packets.

Status	Code	Trigger Condition
BOAT_OK	0	None of the conditions below are true
BOAT_TILT_WARN	1	Max(pitch , roll) > 20° — mild tilt
BOAT_TILT_CRIT	2	Max(pitch , roll) > 45° — severe tilt
BOAT_CAPSIZED	3	Max(pitch , roll) > 60° — likely capsized

BOAT_ROUGH_WATER	4	Sum of gyro axes > 50 dps — violent motion
BOAT_STUCK	5	Motors running for >3s with gyro total < 2 dps — not moving

To replace this with different logic (for drones, robots, etc.), edit `IMU_CheckSafety()` only. The rest of the alert pipeline remains unchanged.

8.4 Magnetometer Calibration (Required for Accurate Heading)

The AK09916 magnetometer reads the local magnetic field. Near motors, batteries, and ESCs, the field is distorted. Without calibration, the heading can be off by 10–40 degrees. Calibration measures this fixed distortion (hard-iron offset) and corrects it.

Calibration Procedure

7. Power on the complete system (all motors armed, radio active, battery connected)
8. Take the vehicle to an open area away from large metal structures
9. Enable calibration mode — add a temporary flag to log `mag_raw[]` via `printf()`
10. Slowly rotate the vehicle through a complete 360° on a flat surface
11. Record the minimum and maximum values on each axis (X, Y, Z)
12. Calculate the hard-iron offset: $\text{offset} = (\text{max} + \text{min}) / 2$
13. Add the offsets to `IMU_Process()` as shown below
14. Verify: with vehicle pointing North, `imu_heading` should read approximately 0°

Applying Hard-Iron Calibration Offsets

```
// Example calibration result (your values will differ):
#define MAG_OFFSET_X    -120
#define MAG_OFFSET_Y     340
#define MAG_OFFSET_Z     85

// In IMU_Process(), replace the mag_ut[] lines with:
mag_ut[0] = (float)(mag_raw[0] - MAG_OFFSET_X) * 0.15f;
mag_ut[1] = (float)(mag_raw[1] - MAG_OFFSET_Y) * 0.15f;
mag_ut[2] = (float)(mag_raw[2] - MAG_OFFSET_Z) * 0.15f;
```

NOTE: Recalibrate whenever the physical layout of the vehicle changes — moving a battery, adding metal components, or changing motor positions all affect the magnetic field.

9. Troubleshooting

Symptom	Likely Cause	Fix
[IMU] NOT FOUND on boot	Wrong I2C address or no pull-up resistors	Check PB6/PB7 wiring. Verify 4.7k pull-ups on SDA and SCL. Try both AD0 states (GND and 3.3V).
[RADIO] fails after 3 attempts	Radio module M0/M1/AUX pins not connected or wrong UART	Verify USART3 TX→Radio RX and RX←Radio TX. Check M0, M1, and AUX GPIO pin definitions in e220.h. Confirm module is powered at 3.3V.
GPS shows 0 satellites	Antenna blocked or GPS UART not receiving	Move to outdoors with clear sky view. Check PA3 receives data from GPS TX pin at 115200 baud.
Motors do not spin at low joystick	MOTOR_LEFT/RIGHT_START too low for your ESC	Increase MOTOR_LEFT_START and MOTOR_RIGHT_START by 50 μ s increments until motors spin reliably.
Motor stops immediately after starting	MOTOR_TIMEOUT_MS too short or no TX contact	Confirm TX is sending motor commands. Check radio link on TX TFT. Verify MOTOR_TIMEOUT_MS=15000.
Heading always reads wrong direction	Magnetometer not calibrated	Perform the hard-iron calibration in Section 8.4 with all electronics powered on.
Tilt warnings on calm water	TILT_WARN_DEG too low for the hull shape	Increase TILT_WARN_DEG from 20° to 30° or 35° and test again on the water.
TX shows no GPS data after 60s	GPS has no fix (normal in first few minutes)	Wait up to 3 minutes outdoors from cold start. TX will show satellite count increasing.
Bait servo does not move	Servo power missing or wrong TIM channel	Servo needs 5V supply (not 3.3V). Verify TIM2 CH1 is started in MX_TIM2_Init().
BOAT_STUCK status when vehicle is moving	STUCK_GYRO_MIN too high for slow movement	Reduce STUCK_GYRO_MIN from 2.0 to 0.5 dps, or increase STUCK_TIMEOUT_MS.

9.1 Boot Console Checklist

Connect a SWO/ITM debug probe and open a terminal at 2 MHz SWO clock. A healthy boot sequence produces:

Expected Boot Console Output

```
=====
RX BOAT - Production v5.5b
Motor keepalive: ANY TX message
Timeout: 15s (TX dead only)
L start: 1200 R start: 1370
=====

[ESC] Armed at 1000
[LIGHT] PA1+PB5 OFF
```

```
[IMU] OK          ← must appear
[MAG] OK          ← or FAIL if magnetometer not detected
[RADIO] OK        ← must appear

=== RUNNING ===
```

NOTE: If [IMU] NOT FOUND appears, motors and radio still work but IMU safety and heading are disabled. The system enters a degraded mode — safe to test but do not use in water.

9.2 Getting Help

- Open an issue on GitHub with your boot console output and the symptom
- Join the Discord server (link in README) for real-time help
- Include your config.h changes and wiring description in any bug report
- Tag issues with the application type (bait-boat, hovercraft, etc.) for faster routing